

第 15, 16 回 グラフアルゴリズム

執筆者: 安永憲司

1 なぜグラフなのか

様々な問題はグラフの言葉で表すことができる。地図にある地域を塗り分けるには何色あればよいかという問題を考える。地図自体は様々な情報を含んでいるが、塗り分けという問題を解くためには、各地域を頂点で表し、隣接する地域に辺を張ったグラフを考えればよい。各辺の端点が異なる色をもつように、各頂点に色を割り当てることができれば、地域の塗り分けができる。このような頂点への色の割当を**グラフ彩色 (graph coloring)**と呼ぶ。図 1 は富山県の市町村の地図とその隣接関係を表すグラフである。この例に対し、4 色で彩色できることがわかる。実は、任意の平面地図は 4 色で塗り分けることができる。グラフの言葉では、任意の平面グラフは 4 色で頂点彩色できる (**四色定理**)。一方、任意のグラフが与えられたとき、3 色で彩色できるかを判定する問題は **NP 完全 (NP complete)** であり、多項式時間で解くことは難しいと考えられている。

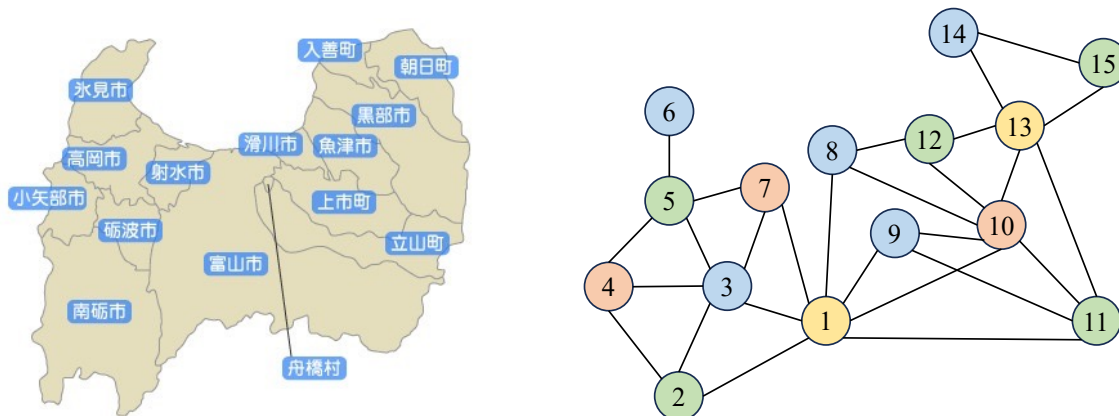


図 1 富山県の市町村とそのグラフ

大学における講義の期末試験の時間割作成問題を考える。試験が必要なすべての講義に対して試験時間を割り当てる必要があるが、なるべく短い期間（少ない時間枠）ですべてを実施したい。ある学生が複数の試験を受ける場合、その講義の試験を同じ時間に割り当てることはできない。この問題はグラフで表すことができる。各試験を頂点に対応させ、ある 2 つの試験を同じ学生が受ける場合には対応する頂点の間に辺を張る。各試験時間枠を一つの色とみなせば、試験時間の割当はグラフ彩色に対応する。図 2 はある大学における講義と試験の制約を表している。この例では、試験時間は 5 コマあれば十分ことがわかる。

病院における手術スケジューリング問題を考える。各手術には担当する医師と手術室が割り当てられているが、医師や手術室は同じ時間に複数の手術を行うことはできない。この制約のもと、なるべく短い期間にすべての手術を完了したい。この問題もグラフで定式化できる。医師と手術室を頂点とし、「ある医師がある手術室で手術を行う」場合に両者を結ぶ辺を張る。つまり、1 つの辺は 1 つの手術を表す。医師同士ならびに手術室同士には辺がないため、**2 部グラフ (bipartite graph)** として表現される。同じ医師または同じ手術室を共有する二つの手術を、同じ時間に実施することはできない。したがって、隣接する辺が

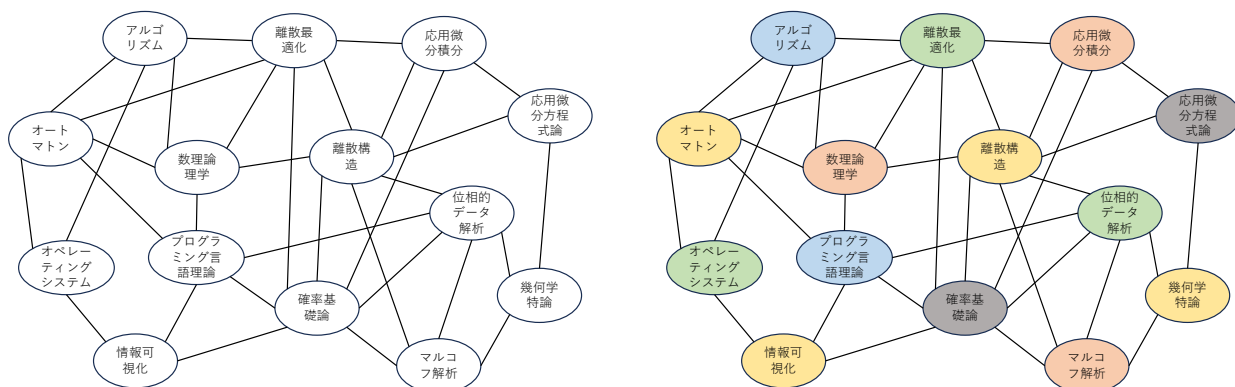


図2 期末試験の時間割作成

異なる色をもつように各辺を彩色する**辺彩色** (edge coloring) に対応する. 図3は, ある大学病院における医師, 手術室, および担当手術を表すグラフの例である. 辺彩色により, 5つの手術スロットがあればすべての手術をスケジュールできることがわかる.

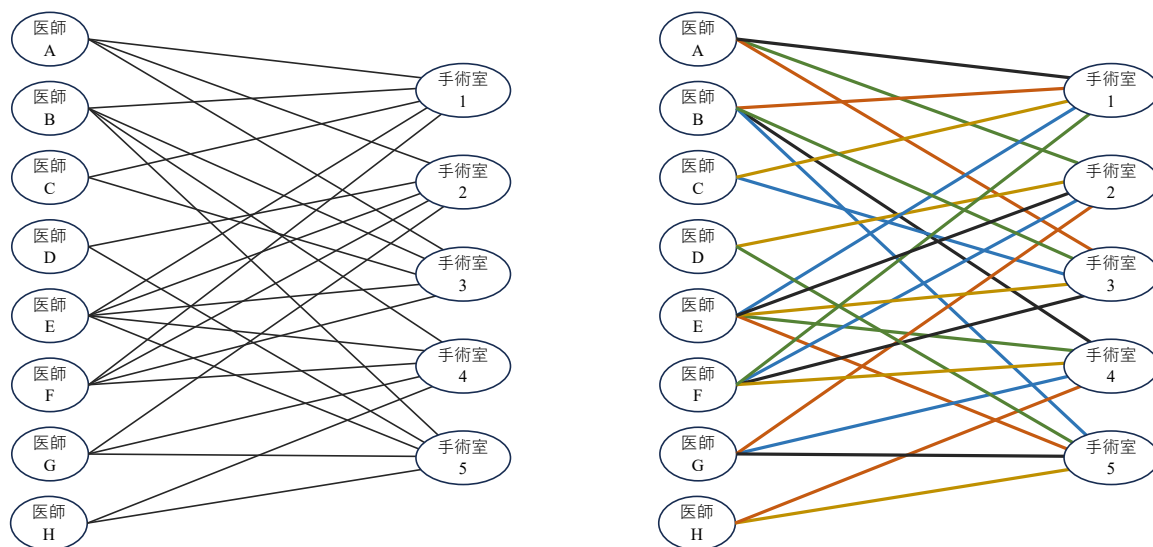


図3 手術スケジューリング

このようにグラフは様々な問題で登場する. ここでは, グラフに対する基本操作を効率的に実行するアルゴリズムについて考える.

グラフ $G = (V, E)$ は, 頂点集合 V と辺集合 E からなる. 有向グラフでは, 各辺は順序対 $(u, v) \in V \times V$ で表され, u から v への辺を意味する. 無向グラフでは, 各辺は順序なし対 $\{u, v\}$ で表されるが, 慣例として (u, v) と書き, (u, v) と (v, u) は同じ辺を表すものとする.

自己ループと多重辺をもたないグラフを**単純グラフ** (simple graph) と呼ぶ. 以下では, 特に断らない限り, 単純グラフを考える.

1.1 グラフの表現

グラフ $G = (V, E)$ の隣接行列 $A = (a_{ij})$ とは, $V = \{v_1, \dots, v_n\}$ のとき,

$$a_{ij} = \begin{cases} 1 & (v_i, v_j) \in E, \\ 0 & \text{それ以外} \end{cases}$$

で定められる $n \times n$ 行列である. 無向グラフの場合には $a_{ij} = a_{ji}$ であるため, 隣接行列は対称行列になる.

隣接行列を用いると, 指定された二頂点の間に辺が存在するかを定数時間で確認できる. 一方で, 必要な領域は $O(|V|^2)$ である.

もう一つの表現方法は隣接リストである. 各頂点 u に対して, $(u, v) \in E$ を満たす頂点 v を並べたリストをもつ. 有向グラフでは各辺は一つのリストに現れ, 無向グラフでは各辺は二つのリストに現れる. したがって, 必要な領域は $O(|V| + |E|)$ である. 一方, 指定された辺 (u, v) が存在するかを調べるには, 一般に u の隣接リストを調べる必要があり, 定数時間ではない. 図 4 は隣接表現の例である.

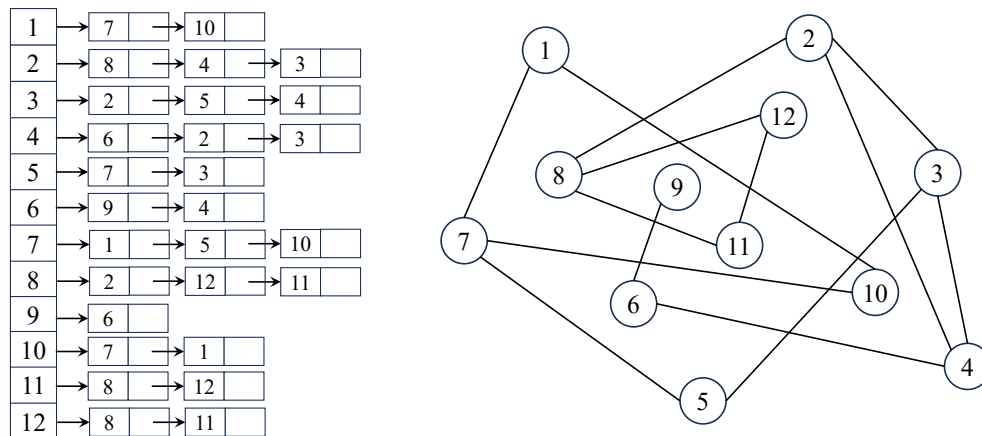


図 4 グラフの隣接リスト表現

辺数 $|E|$ が $|V|^2$ と同程度のとき, グラフは**密** (*dense*) であるという. 一方, $|E|$ が $|V|^2$ に比べて十分小さいとき, グラフは**疎** (*sparse*) であるという. 特に, $|E| = O(|V|)$ であるグラフは疎グラフの典型例である.

クイズ 1.1. 頂点数が n , 辺数が m のグラフを考える. 次の各操作には, 隣接行列と隣接リストのどちらが適しているか答えよ.

- (a) 指定された二頂点の間に辺があるかを何度も判定する.
- (b) 疎グラフのすべての辺を順に調べる.

2 無向グラフにおける深さ優先探索

グラフの最も基本的な問題として, 「与えられた頂点からグラフのどの部分に到達可能 (reachable) であるか」というものがある. グラフが隣接リストで与えられ, ある頂点 v から出発して, どの頂点に到達可能であるかを調べる状況を考える. 隣接リストであるため, ある頂点にいるとき, その隣接頂点のリスト

だけが与えられる。この問題は、迷路を探索することに似ている。うまく移動しないと同じ場所を行ったり来たりすることになり、移動可能な場所をうまく見つけられないことになる。

迷路の古典的な探索方法として、長い糸とチョークを使う方法がある。分岐点では、すでに調べた方向にチョークで印をつけ、糸を使って一つ前の分岐点に戻れるようにする。アルゴリズムでは、チョークの印を各頂点が訪問済みかどうかを表す情報として実現し、糸の役割をスタックによって実現する。新しい頂点へ進むときにその頂点をスタックに積み、探索を終えたときにスタックから取り除けばよい。

```

1  /*******
2  手続き explore( G, v )
3  入力: グラフ G = (V, E) と頂点 v
4  出力: v から到達可能な頂点 u に対して visited(u) = TRUE
5  *****/
6  void explore ( G, v ) {
7      visited(v) = TRUE;
8      previsit(v);
9      for ( v の隣接リストに含まれる各頂点 u について ) {
10         if ( visited(u) == FALSE ) explore(G, u);
11     }
12     postvisit(v);
13 }

```

図5 与えられた一点からの到達可能な全頂点を見つける

スタックの機能は再帰により実現できる。アルゴリズムは図5である。最初に、すべての頂点 u について $visited(u) = FALSE$ と初期化しておく。previsit と postvisit はオプションであり、ある頂点を最初に見つけたときと、その頂点を最後に離れるときに実行される。

手続き explore が正しく動作することを確認しよう。まず、explore が訪れる頂点は、すべて始点 v から到達可能である。問題は、 v から到達可能な頂点をすべて訪れるかどうかである。もし v から到達可能な頂点 u を見逃していたとする。 v から u へのパスを一つ固定し、そのパス上で訪問された最後の頂点を z とする。 u は訪問されていないため、 z の次には訪問されていない頂点 w が存在する。しかし、 z を訪れたとき、explore は z のすべての隣接頂点を調べるため、 w も訪問される。これは矛盾である。

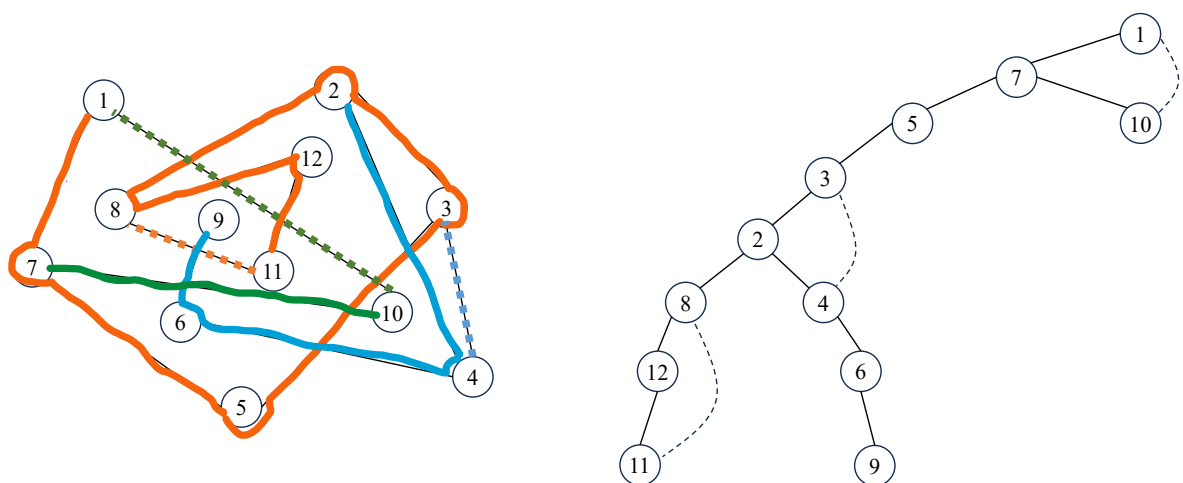


図6 explore の実行結果

新しい頂点を初めて訪れるときに使われた辺を**木辺** (*tree edge*) と呼ぶ。木辺全体は、探索の始点を根とする**深さ優先探索木** (*DFS tree*) を構成する。

図 4 のグラフに対する実行結果が図 6 である。左のグラフは探索の過程を表し、右のグラフでは、頂点 1 から **explore** を実行したときの木辺を実線で、それ以外の辺を点線で示している。無向グラフでは、木辺でない辺は、DFS 木における頂点とその先祖を結ぶ。このような辺を**戻り辺** (*back edge*) と呼ぶ。ただし、親子を結ぶ木辺を逆向きに調べた場合は、戻り辺とは数えない。

手続き **explore** は、始点から到達可能な頂点だけを訪れる。まだ訪問されていない頂点があれば、その頂点から新たに **explore** を実行する必要がある。それを実現したのが図 7 の**深さ優先探索** (*depth-first search (DFS)*) である。

```

1  void dfs ( G ) {
2      for ( V に属するすべての v について ) {
3          visited(v) = FALSE;
4      }
5      for ( V に属するすべての v について ) {
6          if ( visited(v) == FALSE ) {
7              explore(G, v);
8          }
9      }
10 }

```

図 7 深さ優先探索

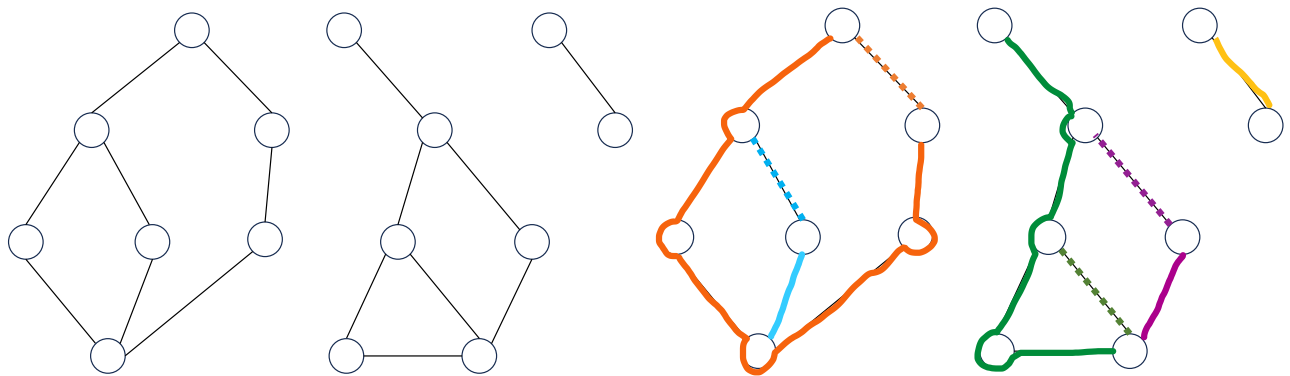


図 8 深さ優先探索の実行結果

実行時間を解析しよう。各頂点 v について、**explore**(G, v) が呼び出されるのは高々一度である。したがって、訪問済みの印を付け、**previsit**(v) と **postvisit**(v) を実行する時間の合計は $O(|V|)$ である。

また、各頂点 v の隣接リストを一度ずつ調べる。無向グラフでは各辺が両端点の隣接リストに一度ずつ現れるため、すべての隣接リストの長さの合計は $2|E|$ である。したがって、隣接頂点を調べる時間の合計は $O(|E|)$ である。

以上より、深さ優先探索の実行時間は

$$O(|V| + |E|)$$

であり、隣接リスト表現の入力サイズに対して線形時間である。

2.1 無向グラフの連結性

無向グラフが**連結** (*connected*) であるとは、任意の二頂点の間にパスが存在することをいう。また、互いにパスで結ばれている頂点をまとめて得られる極大な頂点集合を**連結成分** (*connected component*) と呼ぶ。

ある頂点 v から `explore` を実行すると、 v を含む連結成分の頂点をちょうどすべて訪れる。したがって、深さ優先探索 `dfs` の二つ目の `for` ループで新たに `explore` が呼び出されるたびに、一つの新しい連結成分が見つかる。このことから、深さ優先探索によってグラフが連結かどうかを判定でき、さらに、各頂点にその属する連結成分の番号を割り当てることができる。

各頂点に連結成分の番号を割り当てするには、図 7 の深さ優先探索を少し変更すればよい。変数 `cc` を 0 に初期化し、まだ訪問されていない頂点から新たに `explore` を呼び出すたびに `cc` を 1 増やす。また、各頂点を初めて訪れたとき、その時点の `cc` の値を `ccnum` に記録する。

```
1  void previsit ( v ) {
2      ccnum[v] = cc;
3  }
4
5  void dfs_with_ccnum ( G ) {
6      for ( V に属するすべての v について ) {
7          visited(v) = FALSE;
8      }
9
10     cc = 0;
11     for ( V に属するすべての v について ) {
12         if ( visited(v) == FALSE ) {
13             cc = cc + 1;
14             explore(G, v);
15         }
16     }
17 }
```

図 9 の左側は、グラフ上に深さ優先探索の結果を示したものであり、右側は、得られた木辺を木の形に並べ直したものである。各連結成分に対して一つの深さ優先探索木が得られ、それら全体を**深さ優先探索森** (*DFS forest*) と呼ぶ。

実行結果より、

$\{1, 2, 3, 4, 11, 12, 13\}, \{5, 6, 7, 10, 14, 15\}, \{8, 9\}$

という 3 つの連結成分から成るグラフであることがわかる。

次に、連結成分の番号付けとは別に、各頂点を最初に訪れた時刻と、その頂点からの探索を終えた時刻を記録する。`clock` を 1 に初期化し、`previsit` と `postvisit` を次のように定めて、改めて深さ優先探索を実行する。右側の深さ優先探索森の各頂点 v のそばにある数字は、このとき得られる $\text{pre}(v), \text{post}(v)$ の値である。

```
1  void previsit ( v ) {
2      pre[v] = clock;
3      clock = clock + 1;
4  }
5  void postvisit ( v ) {
```

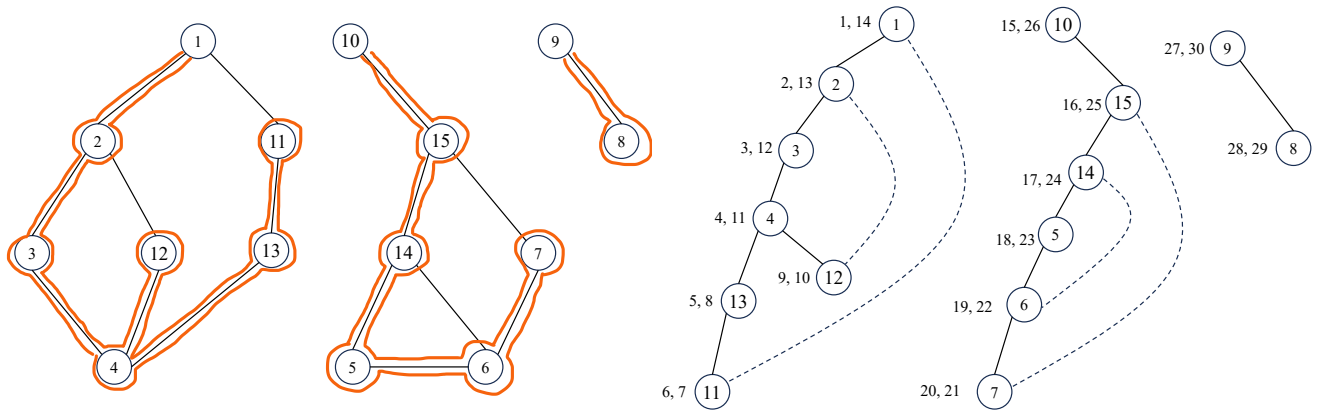


図9 無向グラフと深さ優先探索森

```

6   post[v] = clock;
7   clock = clock + 1;
8   }

```

このように定めた $\text{pre}(v), \text{post}(v)$ について、以下が成り立つ。

命題 2.1. 任意の二頂点 u, v に対し、二つの区間

$$[\text{pre}(u), \text{post}(u)], \quad [\text{pre}(v), \text{post}(v)]$$

は互いに交わらないか、一方が他方に含まれる。

区間 $[\text{pre}(u), \text{post}(u)]$ は、頂点 u に対する再帰呼出しがまだ終了しておらず、その情報がスタックに残っている期間に対応する。したがって、後入れ先出しであるスタックの性質から、二つの区間は互いに交わらないか、一方が他方に含まれる。

特に、頂点 v が DFS 木において頂点 u の子孫であるための必要十分条件は、

$$\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$$

である。

クイズ 2.1. 同じグラフに対して深さ優先探索を実行する。グラフが隣接リストで与えられる場合と隣接行列で与えられる場合について、それぞれの実行時間を答えよ。

クイズ 2.2. DFS 木において頂点 v が頂点 u の子孫であるとする。次の大小関係を正しく並べよ。

$$\text{pre}(u), \quad \text{post}(u), \quad \text{pre}(v), \quad \text{post}(v)$$

3 有向グラフにおける深さ優先探索

深さ優先探索は、有向グラフに対してもそのまま適用することができる。図 10 は有向グラフとそれに対する深さ優先探索の実行結果である。右側の探索木において、頂点 1 は根 (root) であり、それ以外の頂点はすべて子孫 (descendant) である。頂点 9 は頂点 4, 5, 6, 7, 8 を子孫にもち、それらの先祖 (ancestor) である。頂点 5 は頂点 6 の親 (parent) であり、6 は 5 の子 (child) である。

辺については、無向グラフの場合よりさらに細かく分類する。

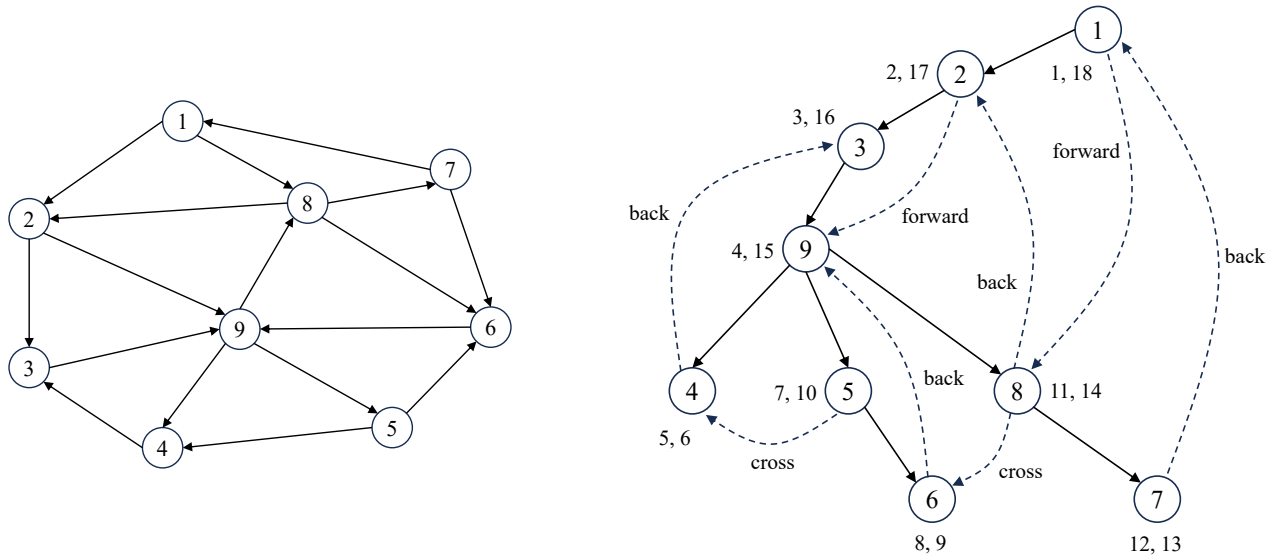


図 10 有向グラフとそれに対する深さ優先探索

- **木辺** (*tree edge*) : 新しい頂点を初めて訪れるときに使われた辺
- **進み辺** (*forward edge*) : DFS 木において、頂点からその子ではない子孫へ向かう辺
- **戻り辺** (*back edge*) : DFS 木において、頂点からその先祖へ向かう辺
- **渡り辺** (*cross edge*) : DFS 森において、先祖・子孫の関係にない頂点へ向かう辺

辺 (u, v) の種類は、 $\text{pre}(u), \text{post}(u), \text{pre}(v), \text{post}(v)$ の大小関係から、次のように判定できる。括弧は、各頂点に対する探索が開始してから終了するまでの区間を表す。

- 木辺または進み辺：

$$\begin{bmatrix} \text{ } \\ u \end{bmatrix} \begin{bmatrix} \text{ } \\ v \end{bmatrix} \begin{bmatrix} \text{ } \\ v \end{bmatrix} \begin{bmatrix} \text{ } \\ u \end{bmatrix}$$

すなわち、

$$\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u).$$

- 戻り辺：

$$\begin{bmatrix} \text{ } \\ v \end{bmatrix} \begin{bmatrix} \text{ } \\ u \end{bmatrix} \begin{bmatrix} \text{ } \\ u \end{bmatrix} \begin{bmatrix} \text{ } \\ v \end{bmatrix}$$

すなわち、

$$\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v).$$

- 渡り辺：

$$\begin{bmatrix} \text{ } \\ v \end{bmatrix} \begin{bmatrix} \text{ } \\ v \end{bmatrix} \begin{bmatrix} \text{ } \\ u \end{bmatrix} \begin{bmatrix} \text{ } \\ u \end{bmatrix}$$

すなわち、

$$\text{pre}(v) < \text{post}(v) < \text{pre}(u) < \text{post}(u).$$

ただし、 pre, post の値だけでは、木辺と進み辺を区別できない。木辺であるかどうかを判定するには、 v が辺 (u, v) を使って初めて発見されたかを確認する必要がある。

3.1 有向非巡回グラフ

グラフにおいて、頂点 $v_0, v_1, \dots, v_k$ が互いに異なり、

$$v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k \rightarrow v_0$$

という辺が存在するとき、これを**単純閉路** (*simple cycle*) と呼ぶ。本資料では、単純閉路を単に**閉路** (*cycle*) と呼ぶ。閉路のないグラフは**非巡回** (*acyclic*) と呼ばれる。深さ優先探索により、線形時間で非巡回性の検査ができる。

命題 3.1. 有向グラフは閉路をもつとき、かつそのときに限り深さ優先探索で戻り辺をもつ。

証明. 深さ優先探索で戻り辺 (u, v) があるとき、 v から u へのパスもあるため、これらを組みあわせると閉路になる。

次に、グラフが閉路

$$v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k \rightarrow v_0$$

をもつとする。この閉路上の頂点のうち、深さ優先探索で最初に発見された頂点を v_i とする。閉路上の他のすべての頂点は v_i から到達可能であるため、 v_i に対する探索が終了するまでに発見され、DFS 木において v_i の子孫になる。閉路上で v_i の直前にある頂点を v_j とする。頂点 v_j は DFS 木において v_i の子孫であるため、辺 (v_j, v_i) は子孫から先祖への辺であり、戻り辺である。□

有向非巡回グラフ (*directed acyclic graph, DAG*) は頻出のグラフである。例えば、多くのタスクをこなす必要があり、あるものは別のタスクが完了してからでないと行えないとする。朝起きたあと、布団から出て、服を着替えて、シャワーを浴びて、食前のストレッチをして、朝ご飯を食べて、歯を磨くなど。このとき、どの順番でタスクをこなせばよいかを考えてみよう。

各タスクを頂点で表し、タスク u を v の前に完了する必要があるときに辺 (u, v) を引く。例えば、先程の各タスクを頂点 A から F まで以下のように割り当て、

A : 布団から出る, B : 服を着替える, C : シャワー, D : 食前ストレッチ, E : 朝食, F : 歯磨き

図 11 左のようなグラフをつくる。もしこのグラフに閉路があったとすると、どの順番でもうまくいかない。したがって、DAG であれば、すべての制約を満たすように頂点を順に並べられることを期待する。このグラフであれば、 (A, D, C, E, B, F) というのが一つの並べ方である。

有向グラフ $G = (V, E)$ の**トポロジカル順序** (*topological ordering*)・**トポロジカルソート** (*topological sort*) とは、 G の全頂点を一列に並べたものであり、 $(u, v) \in E$ ならば、 u が v より前に現れるものをいう。ここでいう順序は、数値や文字列の大小順ではなく、辺が表す先行関係を満たす順序である。トポロジカル順序を求めることを**トポロジカルソート** (*topological sorting*) と呼ぶ。上記はトポロジカルソートの問題であった。

任意の DAG に対して、トポロジカルソートが存在することがわかる。深さ優先探索を実行し、 $\text{post}(v)$ の降順に出力すればよい。なぜならば、出力した順序がトポロジカルソートでないのは辺 (u, v) が存在して、 $\text{post}(u) < \text{post}(v)$ を満たすときであるが、辺の分類より、辺 (u, v) について $\text{post}(u) < \text{post}(v)$ となるのは、 (u, v) が戻り辺の場合だけである。命題 3.1 より、DAG に対する深さ優先探索は戻り辺をもたないため、そのような辺 (u, v) は存在しない。したがって、以下が成り立つ。

命題 3.2. DAG $G = (V, E)$ に対し、 $(u, v) \in E$ ならば、深さ優先探索の結果、 $\text{post}(u) > \text{post}(v)$ である。

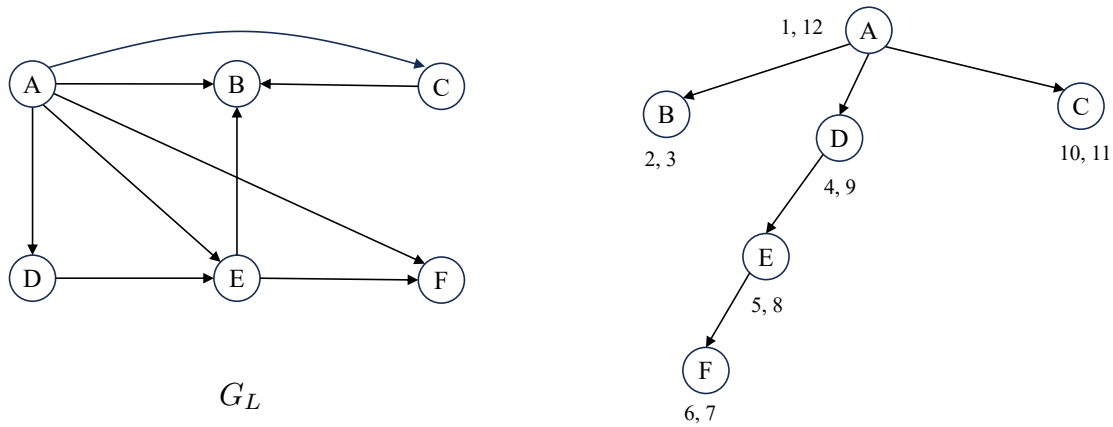


図 11 有向非巡回グラフとそれに対する深さ優先探索

以上より，次の命題が成り立つ．

命題 3.3. 有向グラフ G について，以下は等価である．

- (a) G は閉路をもたない．
- (b) G はトポロジカルソートをもつ．
- (c) G の深さ優先探索には戻り辺がない．

DAG の頂点の中で $\text{post}(u)$ が最大の頂点 u は，トポロジカルソートの先頭であり，逆に，これが最小の頂点 v は最後にある．DAG では最大の頂点 u に入る辺 (incoming edge) は存在せず，最小の頂点 v から出る辺 (outgoing edge) も存在しない．この性質を利用してトポロジカルソートを見つける別のアルゴリズムが以下の通りである．

1. 入次数が 0 である頂点 u を一つ選び， u を出力する．
2. u と， u から出るすべての辺をグラフから削除する．
3. 頂点がなくなるまで繰り返す．

入次数を配列に記録し，入次数が 0 になった頂点をキューに格納すれば，このアルゴリズムも $O(|V|+|E|)$ 時間で実行できる．

クイズ 3.1. 有向グラフの DFS で辺 (u, v) について

$$\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$$

が成り立つとする．この辺の種類を答えよ．

4 強連結成分

有向グラフの連結性を定義したい．無向グラフの場合，頂点がつながっているもの同士を連結成分とすればよかった．有向グラフでは，頂点 u から v へのパスがあったとしても，その逆が成り立つとは限らない．そこで，有向グラフ $G = (V, E)$ の頂点 $u, v \in V$ が**強連結** (strongly connected) であるとは， u から v へのパスと v から u へのパスがともに存在するときと定める．この関係によって頂点集合 V を分割して得られる極大な頂点集合を**強連結成分** (strongly connected component) と呼ぶ．さらに，各強連結成分を

一つの頂点にまとめたグラフを考える．異なる強連結成分 C, C' について， C の頂点から C' の頂点への辺が存在するとき， C から C' へ辺を張る．このグラフを**縮約グラフ** (*condensation graph*) と呼ぶ．本資料では**メタグラフ** (*meta graph*) とも呼ぶことにする．

縮約グラフは必ず DAG である．もし複数の強連結成分が縮約グラフ上で閉路を構成すれば，それらの成分の間を互に行き来できるため，一つの強連結成分にまとめられるからである．

命題 4.1. 有向グラフの縮約グラフは DAG である．

したがって，任意の有向グラフは，各強連結成分の内部構造と，それらを結ぶ DAG としての構造に分けて捉えることができる．縮約グラフは DAG なので，その頂点である強連結成分をトポロジカル順序に従って並べることもできる．

有向グラフを強連結成分に分割するアルゴリズムについて考えよう．以下の命題を利用する．

命題 4.2. 頂点 u から `explore` を開始すると， u から到達可能なすべての頂点を訪れたあとにちょうど停止する．

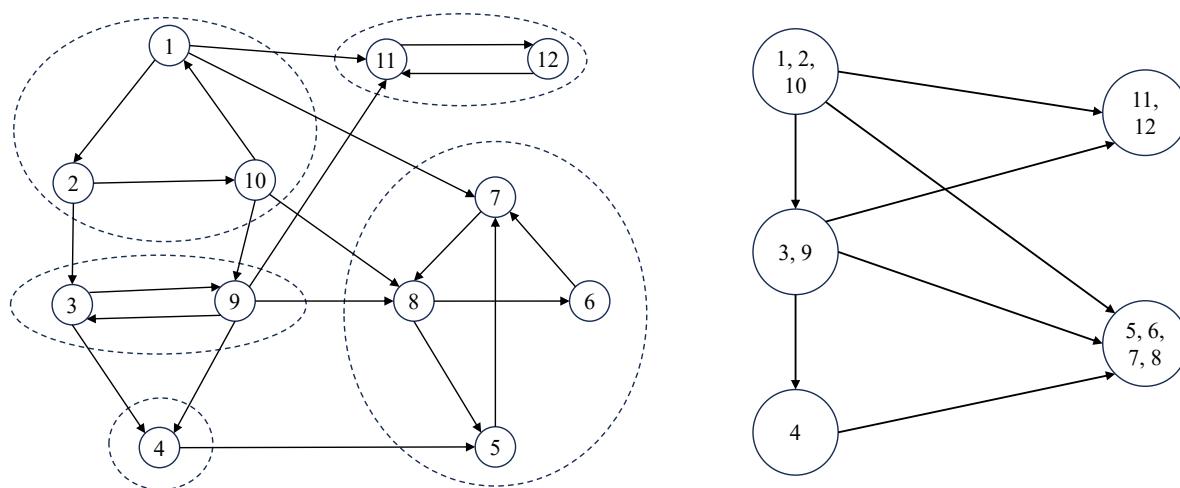


図 12 有向グラフにおける強連結成分とそのメタグラフ

メタグラフの**終点** (*sink*) (出る辺をもたない頂点) に対応する強連結成分に属する頂点から `explore` を開始すれば，その強連結成分の頂点だけをちょうどすべて訪れる．終点からは他の強連結成分へ向かう辺がないためである．図 12 のメタグラフには終点が 2 つあり，例えば頂点 5 から `explore` を始めると，強連結成分を見つけて停止する．この考えを利用して，すべての強連結成分を見つける効率的なアルゴリズムを構成しよう．

問題を

- (A) メタグラフの終点に対応する強連結成分の頂点を見つける方法
- (B) 一つの強連結成分を見つけたあと，残りの強連結成分を見つける方法

の二つに分けて考える．

問題 (A) を解くために，メタグラフの**始点** (*source*) (入る辺をもたない頂点) に関する以下の命題を使う．

命題 4.3. 深さ優先探索で $\text{post}(v)$ が最大となる頂点 v を含む強連結成分は，メタグラフの始点である．

この命題は、以下のより一般的な命題から導かれる。

命題 4.4. 強連結成分 C, C' に対し、 C の頂点から C' の頂点への辺があるとき、深さ優先探索のあとの C における post の最大値は、 C' における post の最大値よりも大きい。

証明. 深さ優先探索が、 C' より先に C の頂点を初めて訪れたとする。 C の頂点を初めて訪れたときに開始された再帰的な探索を考える。 C は強連結であり、さらに C から C' への辺があるため、この探索が終了するまでに C と C' のすべての頂点が訪問される。したがって、

$$\max_{v \in C} \text{post}(v) > \max_{v \in C'} \text{post}(v)$$

である。

次に、 C より先に C' の頂点を初めて訪れたとする。 C' から C へのパスは存在しない。もし存在すれば、 C から C' への辺と合わせて C と C' が同じ強連結成分になってしまうためである。したがって、 C' から始まった探索は C を訪れる前に終了する。深さ優先探索全体では、その後に C の頂点も訪問されるため、この場合にも

$$\max_{v \in C} \text{post}(v) > \max_{v \in C'} \text{post}(v)$$

が成り立つ。 □

命題 4.4 より、強連結成分をトポロジカル順序で並べる方法がわかる。各強連結成分 C に対して

$$\text{post}(C) = \max_{v \in C} \text{post}(v)$$

と定める。強連結成分を $\text{post}(C)$ の降順に並べると、メタグラフのトポロジカルソートが得られる。

命題 4.3 は、メタグラフの終点ではなく始点を見つけているため、問題 (A) の逆の問題を解いている。ここで、グラフ G の逆グラフ G^R として、すべての辺の向きを反対にしたものを考えると、 G^R は G と同じ強連結成分をもつ。この G^R に対し深さ優先探索を行い、 post が最大の頂点 v を見つけると、 v を含む強連結成分は、 G のメタグラフにおける終点である。これで問題 (A) は解けた。

次に問題 (B) を考える。 G^R のメタグラフでは、 G の終点と始点が入れ替わる。したがって、 G^R に対する深さ優先探索で得られた post 番号の降順は、 G のメタグラフの終点側から強連結成分を調べる順序を与える。

この順に元のグラフ G の頂点を調べ、まだ訪問されていない頂点から `explore` を実行すると、一つの強連結成分だけを訪れる。一度訪れた頂点を飛ばしてこの処理を続ければ、すべての強連結成分を順に取り出すことができる。

強連結成分を求めるアルゴリズムは以下の通りである。

1. 逆グラフ G^R に対して深さ優先探索を実行し、各頂点の post 番号を求める。
2. すべての頂点を未訪問に戻す。
3. G^R で得られた post 番号の降順に頂点 v を調べる。 v が未訪問なら、元のグラフ G 上で v から `explore` を実行する。この探索で訪れた頂点全体が一つの強連結成分である。

逆グラフの作成と二回の深さ優先探索はいずれも $O(|V| + |E|)$ 時間で実行できる。したがって、全体の実行時間も $O(|V| + |E|)$ である。

図 12 のグラフに適用する。ステップ 1 で G^R に対して深さ優先探索を実行すると、 post 番号の降順は

11, 12, 5, 8, 7, 6, 4, 3, 9, 1, 10, 2

となる。この順に元のグラフ G 上で探索すると、

$$\{11, 12\}, \{5, 6, 7, 8\}, \{4\}, \{3, 9\}, \{1, 2, 10\}$$

の順に強連結成分が見つかる。

クイズ 4.1. 有向グラフ G のすべての辺を逆向きにして得られる逆グラフ G^R を考える。 G と G^R の強連結成分は同じである。正しければ理由を説明し、誤っていれば反例を示せ。

クイズ 4.2. 有向グラフの縮約グラフにおいて、異なる二頂点 C, C' の間に C から C' へのパスと C' から C へのパスがともに存在することはあるか。理由とともに答えよ。

5 最短パスと幅優先探索

深さ優先探索は、ある頂点から到達可能なすべての頂点を見つけることができる。しかし、そこで見つかったパスは「よいもの」とは限らない。例えば、図 9 の深さ優先探索では、頂点 1 から 11 へのパスとして、 $(1, 2, 3, 4, 13, 11)$ が見つかったが、頂点 11 に辿り着くには、 $(1, 11)$ で十分である。以降ではグラフにおいて**最短パス** (*shortest path*) をを見つける方法について考える。パスの**長さ** (*length*) を、そのパスに含まれる辺の数と定める。二頂点間の**距離** (*distance*) を、その二頂点を結ぶ最短パスの長さとして定める。二頂点の間にパスが存在しない場合、その距離を ∞ と定める。

5.1 幅優先探索

与えられた頂点 s から距離の小さい頂点を順に見つける単純な方法がある。距離 0 の点として s があり、 s との距離が 1 の頂点は s に隣接する頂点である。さらに距離 2 の頂点は、距離 1 の頂点に隣接する頂点でまだ見つからない頂点である。これを繰り返せば到達可能なすべての頂点を距離の順に見つけられる。

幅優先探索 (*breadth-first search, BFS*) は、距離の小さい頂点から順に探索するアルゴリズムである (図 13)。データ構造としてキュー Q を利用する。最初は、距離 0 の頂点 s だけを Q に格納する。頂点 u をキューから取り出したとき、まだ発見されていない隣接頂点 v をキューの末尾に追加し、

$$\text{dist}(v) = \text{dist}(u) + 1$$

と定める。キューから取り出される頂点の距離は単調非減少となるため、頂点は始点 s からの距離が小さい順に処理される。

アルゴリズム `bfs` の実行結果が図 14 である。頂点 v を初めて発見したときに使った辺 (u, v) を、 v の親への辺として記録する。このような辺全体は、 s から到達可能な頂点を頂点集合とする木を構成する。これを**幅優先探索木** (*breadth-first search tree*) と呼ぶ。幅優先探索木における s から各頂点へのパスは最短パスであるため、この木は**最短パス木** (*shortest-path tree*) でもある。

アルゴリズムの正しさと効率

幅優先探索では、キューから取り出される頂点の距離は単調非減少である。実際、距離 d の頂点を処理したときに新たに追加される頂点の距離は $d + 1$ であり、キューの末尾に追加されるからである。

次に、頂点 v を初めて発見した時点で、 $\text{dist}(v)$ が s から v への最短距離に等しいことを示す。 s からの最短距離について帰納法を用いる。始点 s については明らかである。 s から v への最短距離が $d + 1$ であるとし、最短距離が高々 d である頂点について主張が成り立つと仮定する。 s から v への最短パス上で v

```

1  /*****
2  手続き bfs( G, s )
3  入力：グラフ  $G = (V, E)$  と頂点  $s$  ( $G$  は有向でも無向でも)
4  出力： $s$  から到達可能な頂点  $u$  に対して  $dist(u) = (s \text{ から } u \text{ の距離})$ 
5  *****/
6  void bfs ( G, s ) {
7      for ( すべての頂点  $u$  について )  $dist(u) = \infty$ ;
8       $dist(s) = 0$ ;
9       $Q = [s]$ ;
10     while (  $Q$  が空でない限り ) {
11          $u = \text{eject}(Q)$ ;
12         for (  $u$  の隣接リストに含まれる各頂点  $v$  について ) {
13             if (  $dist(v) == \infty$  ) {
14                  $dist(v) = dist(u) + 1$ ;
15                  $\text{inject}(Q, v)$ ;
16             }
17         }
18     }
19 }

```

図 13 幅優先探索 bfs

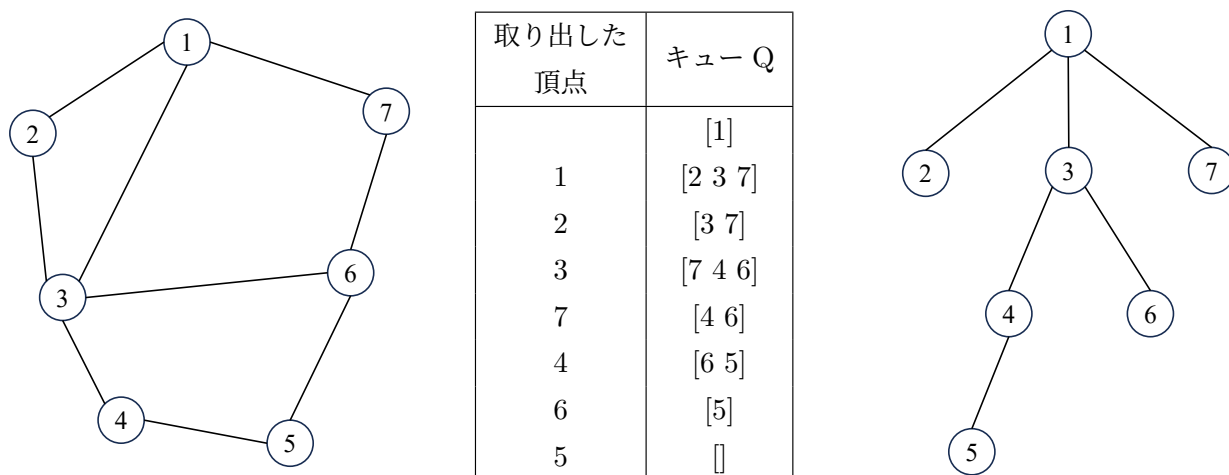


図 14 左のグラフに対し頂点 1 から幅優先探索したときの結果（中央：キューから取り出した頂点と、取り出した後のキューの内容、右：幅優先探索木）

の直前にある頂点を x とすると、 x の最短距離は d である。帰納法の仮定より、 x は距離 d として処理されるため、そのとき v は距離 $d + 1$ 以下として発見される。一方、 v への最短距離は $d + 1$ であるから、 $dist(v) = d + 1$ である。

各頂点は高々一度だけキューに追加され、高々一度だけ取り出されるため、キュー操作にかかる時間の合計は $O(|V|)$ である。また、すべての隣接リストを調べる時間の合計は $O(|E|)$ である。したがって、幅優先探索の実行時間は

$$O(|V| + |E|)$$

である。

クイズ 5.1. 幅優先探索の実行中、キューに格納されている頂点の距離は、高々何種類であるか答えよ。また、それらの距離の関係を答えよ。

クイズ 5.2. 幅優先探索木における始点 s から各頂点へのパスは、必ず元のグラフにおける最短パスである。正しい理由は理由を説明し、誤っていれば反例を示せ。

6 重みつきグラフの最短パスとダイクストラ法

幅優先探索は、与えられた頂点からの最短パスを見つけるアルゴリズムであるが、各辺には同じ「重み」があると考えていた。つまり、隣接する頂点間の距離は 1 であり、ある二頂点間の距離は、その二頂点を結ぶパスの長さの最小値である。より一般的に、各辺に様々な重みがある場合の最短パスを見つける問題も考えられる。例えば、各頂点は富山県の市町村に対応し、隣接する市町村の間に辺があり、その辺ごとに移動コストが決まっており、ある都市からある都市への最短コストの経路を見つけたいというような問題である。グラフ $G = (V, E)$ と各辺 $e \in E$ の重み関数 $w: E \rightarrow \mathbb{R}$ が与えられたとする。重み付きグラフでは、パスの**長さ**を、そのパスに含まれる辺の重みの和と定める。二頂点間の**最短距離** (*shortest distance*) (以下、単に**距離**という) を、その二頂点を結ぶ最短パスの長さとして定める。このとき、与えられた頂点 s から各頂点への距離を求める問題を考える。特にここでは、**非負の重み**をもつグラフを考える。

なお、すべての辺重みが正整数であれば、この問題を幅優先探索によって解くこともできる。図 15 にある通り、重み k の辺を、 $k - 1$ 個の中間頂点を加えた k 本の重み 1 の辺に置き換えればよい。変換後のグラフに対して幅優先探索を実行すれば、元のグラフにおける最短距離が得られる。ただし、この方法では重みの大きさに比例して変換後の頂点数と辺数が増えるため、重みが大きいと効率が悪い。

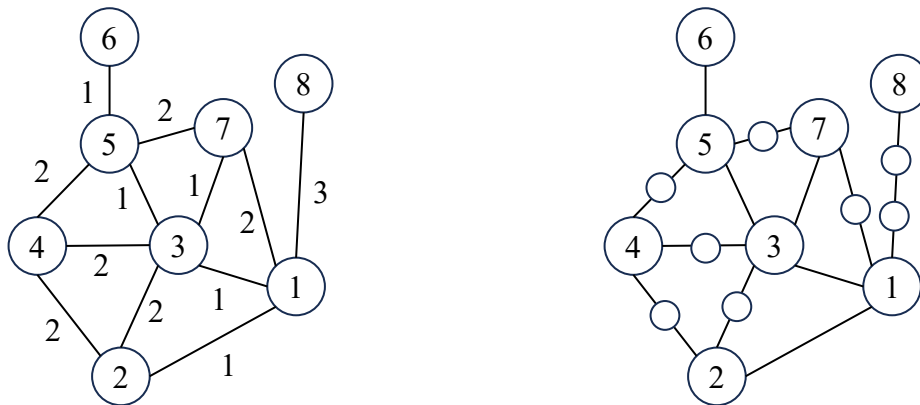


図 15 辺重み 1 のグラフへの変換

この問題を直接解くのが**ダイクストラ法** (*Dijkstra's algorithm*) である。

アルゴリズムでは、各頂点 v について、これまでに見つかった s から v へのパスの長さの最小値を $\text{dist}(v)$ として記録する。この値を v の**暫定距離**と呼ぶことにする。また、最短距離が確定した頂点の集合を R とする。

初めは、

$$R = \emptyset, \quad \text{dist}(s) = 0$$

とし、それ以外の頂点 v について $\text{dist}(v) = \infty$ とする。

各繰り返しでは、 R に属さない頂点のうち、暫定距離が最小の頂点 u を選ぶ。辺重みがすべて非負であるため、この時点で $\text{dist}(u)$ は s から u への最短距離に等しい。そこで、 u を R に加える。

続いて、 u から出る各辺 (u, v) について、 u を経由するパスの方が短ければ、

$$\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + w(u, v)\}$$

と更新する。この更新を緩和 (*relaxation*) と呼ぶ。

```

1  /*****
2  手続き dijkstra( G, w, s )
3  入力：グラフ G = (V, E), 非負重み関数 w, 頂点 s
4  出力：各頂点 u に対して dist(u) = (s から u への距離)
5  *****/
6  void dijkstra ( G, w, s ) {
7      for ( すべての頂点 u について ) dist(u) = ∞;
8      dist(s) = 0;
9      R = [];      // R は最初は空
10     while ( R に属さない頂点 u で dist(u) < ∞ のものが存在する ) {
11         R に属さない頂点 u のうち dist(u) が最小のものを選ぶ;
12         u を R に加える;
13         for ( u の隣接リストに含まれる各頂点 v について ) {
14             if ( dist(v) > dist(u) + w(u,v) ) {
15                 dist(v) = dist(u) + w(u,v);
16             }
17         }
18     }
19 }
```

図 16 ダイクストラ法

6.1 アルゴリズムの正しさと効率

ダイクストラ法の各繰り返しが終了した時点で、次の性質が成り立つ。

1. R に属する各頂点 u について、 $\text{dist}(u)$ は s から u への最短距離に等しい。
2. R に属さない各頂点 v について、 $\text{dist}(v)$ は、中間頂点がすべて R に属する s から v へのパスのうち、長さが最小のものに等しい。そのようなパスがなければ、 $\text{dist}(v) = \infty$ である。

最初の繰り返しでは s が R に加えられ、 s から出るすべての辺が緩和されるため、上の二つの性質が成り立つ。

ある繰り返しの終了時に二つの性質が成り立つと仮定し、次の繰り返しで、 R に属さない頂点のうち暫定距離が最小の頂点 u を選ぶ。 s から u への最短パス P を考え、 P 上で R に属さない最初の頂点を x 、その直前の頂点を y とする。このとき $y \in R$ であり、 y が R に加えられたときに辺 (y, x) は緩和されている。したがって、

$$\text{dist}(x) \leq \text{dist}(y) + w(y, x) \leq \ell_P(s, x) \leq \ell_P(s, u).$$

また、 u は暫定距離が最小なので、

$$\text{dist}(u) \leq \text{dist}(x).$$

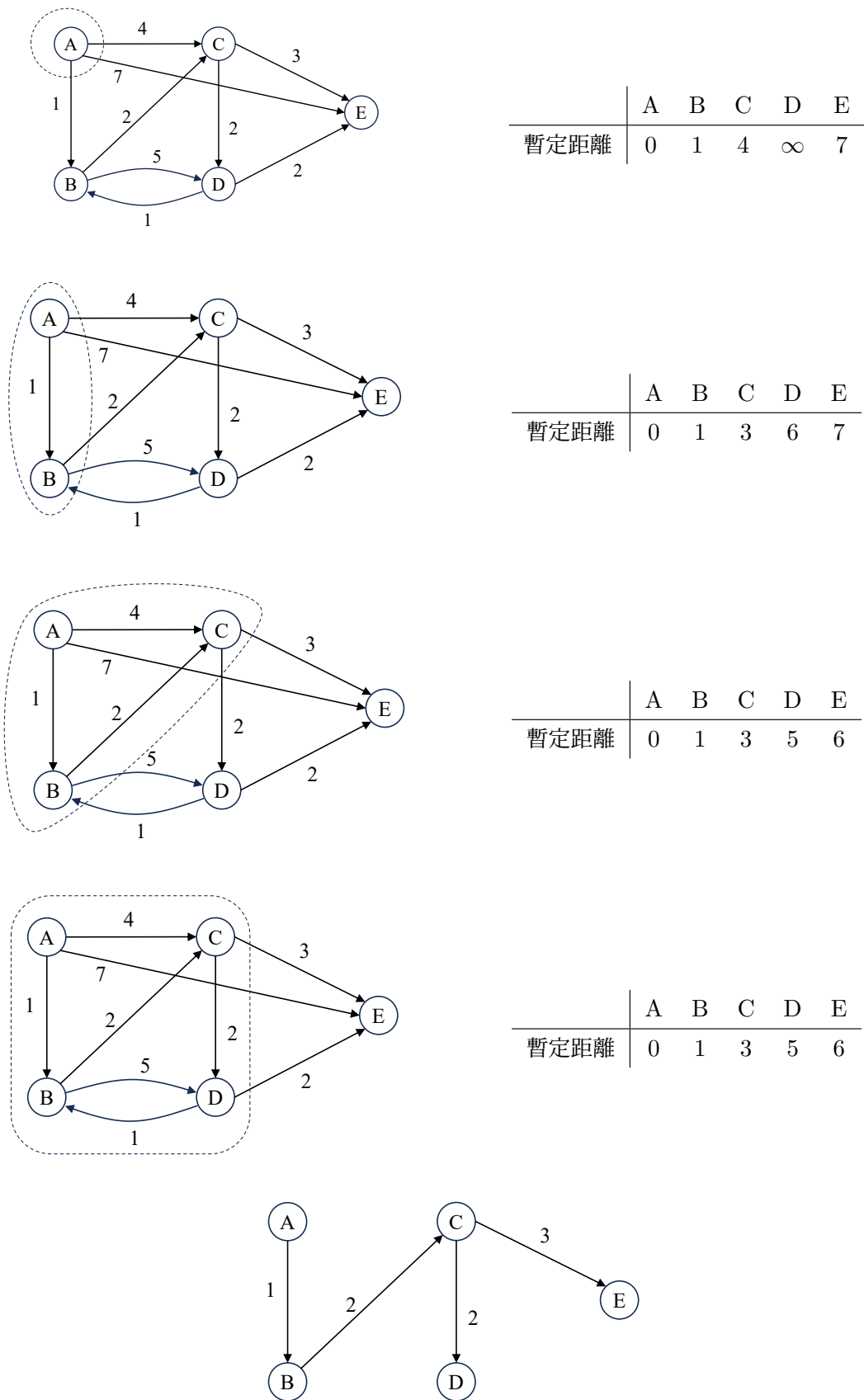


図 17 ダイクストラ法の実行の様子と得られた最短パス木

よって,

$$\text{dist}(u) \leq \ell_P(s, u).$$

一方, $\text{dist}(u)$ は実際に見つかった s から u へのパスの長さであるため, 最短パス P の長さより小さくなることはない. したがって,

$$\text{dist}(u) = \ell_P(s, u)$$

であり, u の最短距離は確定する.

その後, u を R に加え, u から出るすべての辺を緩和することで, 二つ目の性質も保たれる. したがって, 次の繰り返しの終了時にも, 上の二つの性質が成り立つ.

次に実行時間について考える. ダイクストラ法では, R に属さない頂点のうち暫定距離が最小の頂点を繰り返し選び, 隣接頂点の暫定距離を更新する. この処理を効率よく実現するために, 動的集合

$$X = V \setminus R$$

を**優先度付きキュー**で管理する. 各頂点 u の鍵は $\text{dist}(u)$ とする.

必要な操作は以下の通りである.

1. 頂点を優先度付きキューに追加する. (Insert)
2. 鍵が最小の頂点を選び, 取り除く. (DeleteMin)
3. 頂点 v の鍵 $\text{dist}(v)$ を小さくする. (DecreaseKey)

初期化のために Insert を $|V|$ 回実行する. また, DeleteMin は高々 $|V|$ 回, DecreaseKey は各辺の緩和によって高々 $|E|$ 回実行される.

したがって, ダイクストラ法の実行時間は, 優先度付きキューにおけるこれらの操作の実行時間によって決まる. 以下では, 配列とヒープによる実現を考える.

- **配列**: 最も単純なのは, サイズ $|V|$ の配列を使って実現することである. Insert と DecreaseKey は計算量 $O(1)$ であるが, DeleteMin は $O(|V|)$ かかる.

したがって, ダイクストラ法の計算量は $O(|V|^2 + |E|) = O(|V|^2)$ である.

- **ヒープ**: 「データ構造」の回で扱ったヒープを用いる. 各頂点 v を, 鍵 $\text{dist}(v)$ とともにヒープに格納する. さらに, 各頂点 v について, ヒープ内で v が格納されている位置を配列に記録しておけば, DecreaseKey を $O(\log |V|)$ 時間で実行できる. Insert, DeleteMin, DecreaseKey はいずれも $O(\log |V|)$ 時間であるため, ダイクストラ法の実行時間は

$$O((|V| + |E|) \log |V|)$$

である.

ヒープ内の位置を管理しない, より簡単な実装もある. $\text{dist}(v)$ が更新されたとき, 古い要素を削除せず, 新しい要素 $(\text{dist}(v), v)$ をヒープに追加する. 同じ頂点に対応する複数の要素がヒープ内に存在しうるが, 要素 (d, v) を取り出したとき, $d \neq \text{dist}(v)$ ならば古い情報なので無視する.

この方法では, ヒープへの追加と取り出しはそれぞれ $O(|V| + |E|)$ 回であり, 実行時間は

$$O((|V| + |E|) \log(|V| + |E|))$$

である.

ここでは説明しないが, フィボナッチヒープでは, Insert と DecreaseKey をならし $O(1)$ 時間, DeleteMin をならし $O(\log |V|)$ 時間で実行できる. したがって, ダイクストラ法を $O(|E| + |V| \log |V|)$ 時間で実行できる.

クイズ 6.1. 重み 0 の辺を含むグラフに対して、ダイクストラ法を適用することはできるか. 理由とともに答えよ.

7 負重みグラフの最短パス

ダイクストラ法では辺重みが非負である必要があった. グラフの辺に負の重みがある場合はどうなるだろうか. アルゴリズムとして, 距離 $dist$ を全頂点 ∞ に初期化し, より短いパスが見つければ小さい値に更新していた. この更新手続き自体は, 負重みの辺があっても問題ない. しかし, ダイクストラ法では, 選ばれた一頂点から伸びる辺による更新を考えており, この更新方針は負重みがあると不十分であった.

頂点の重複を含まないパスを**単純パス** (*simple path*) と呼ぶ.

負重みがある場合に, 始点 s から頂点 t までの最短パスが $(s, u_1, \dots, u_k, t)$ であったとする. 始点 s から到達可能な負閉路が存在しないと仮定する. このとき, 頂点を重複して通るパスがあれば, その重複部分に含まれる閉路を取り除いてもパスの長さは増加しない. したがって, 各頂点への最短パスとして単純パスを選ぶことができ, その辺数は高々 $|V| - 1$ である.

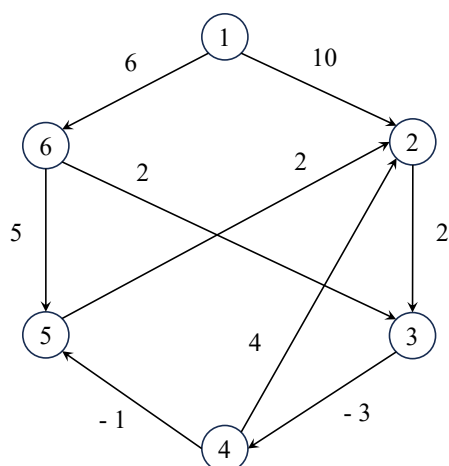
距離の更新が $(s, u_1), (u_1, u_2), \dots, (u_k, t)$ の順に行われれば, s から t への距離が正しく計算される. 辺を調べる順序にかかわらず正しい距離を得るため, すべての辺の緩和を $|V| - 1$ 回繰り返す. これにより, s から各頂点への最短距離を計算できる. このアルゴリズムを**ベルマン - フォード法** (*Bellman-Ford algorithm*) と呼ぶ.

```
1  /*****
2  手続き bellman_ford( G, w, s )
3  入力: 有向グラフ G = (V, E), 重み関数 w, 頂点 s
4  出力: 各頂点 u に対して dist(u) = (s から u への距離)
5  *****/
6  void bellman_ford ( G, w, s ) {
7      for ( すべての頂点 u について ) dist(u) = ∞;
8      dist(s) = 0;
9      for ( 以下を |V|-1 回繰り返す ) {
10         newdist = dist;
11         for ( すべての辺 (u,v) について ) {
12             newdist(v) = min{ newdist(v), dist(u) + w(u,v) };
13         }
14         dist = newdist;
15     }
16 }
```

図 18 ベルマン - フォード法

図 19 は, 左のグラフに対してベルマン-フォード法を実行したときの $dist$ の変化を右に示している.

i 回目の繰り返しを終了した時点で, $dist(v)$ は, 辺数が高々 i である s から v へのパスの長さの最小値に等しい. これは i についての帰納法で示せる. $i = 0$ では, 辺を使わないパスだけを考えるため, $dist(s) = 0$ であり, それ以外の頂点の値は ∞ である. i 回目の繰り返しでは, 辺 (u, v) を最後の辺としてもつパスを考え, 辺数が高々 $i - 1$ である s から u へのパスに (u, v) を加えることで, 辺数が高々 i であるパスの最小値を求めている.



	繰り返し回数					
頂点	0	1	2	3	4	5
1	0	0	0	0	0	0
2	∞	10	10	10	9	6
3	∞	∞	8	8	8	8
4	∞	∞	∞	5	5	5
5	∞	∞	11	11	4	4
6	∞	6	6	6	6	6

図 19 ベルマン-フォード法の実行結果

各繰り返しですべての辺を調べるため、ベルマン-フォード法の実行時間は

$$O(|V||E|)$$

である。

7.1 負の閉路

図 19 のグラフにおいて、辺 $(4, 5)$ の重みが -2 である場合、閉路 $4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4$ の辺重みの和は負になる。このような閉路を**負閉路** (*negative cycle*) と呼ぶ。

負閉路を繰り返し通ることで、パスの長さをいくらでも小さくできる。したがって、始点 s から到達可能な負閉路が存在すると、その負閉路から到達可能な頂点への最短距離は定まらない。

始点 s から到達可能な負閉路を検出するには、 $|V| - 1$ 回の繰り返しの後、さらにもう 1 回、すべての辺を緩和する。このときもいずれかの値が更新されるならば、 s から到達可能な負閉路が存在する。

7.2 DAG における最短パス

DAG には閉路がないため、辺に負の重みがあっても負閉路は存在しない。この性質を利用すると、効率的に最短パスを求めることができる。

命題 7.1. DAG の任意の有向パスにおいて、頂点はトポロジカルソートの順に現れる。

したがって、まず DAG のトポロジカルソートを求め、その順に各頂点から出る辺を緩和すればよい。アルゴリズムは図 20 のとおりである。

トポロジカルソートとすべての辺の緩和は、いずれも $O(|V| + |E|)$ 時間で実行できる。したがって、このアルゴリズムの実行時間は

$$O(|V| + |E|)$$

である。

クイズ 7.1. 始点 s から到達できない負閉路が存在するとき、 s を始点とするベルマン-フォード法は、追加の緩和によってその負閉路を検出するか答えよ。

```

1  /*****
2  手続き dag-shortest-path( G, w, s )
3  入力: DAG G = (V, E), 重み関数 w, 頂点 s
4  出力: 各頂点 u に対して dist(u) = (s から u への距離)
5  *****/
6  void dag-shortest-path ( G, w, s ) {
7      for ( すべての頂点 u について ) dist(u) = ∞;
8      dist(s) = 0;
9      G のトポロジカルソートを求める;
10     for ( トポロジカルソートの順に各頂点 u について ) {
11         if ( dist(u) < ∞ ) {
12             for ( u の隣接リストに含まれる各頂点 v について ) {
13                 dist(v) = min{ dist(v), dist(u) + w(u,v) };
14             }
15         }
16     }
17 }

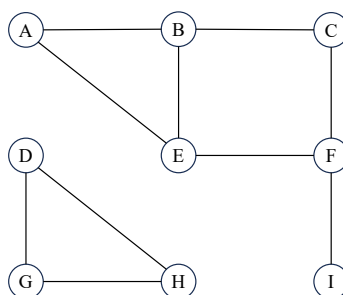
```

図 20 DAG に対する最短パスアルゴリズム

クイズ 7.2. 負の重みをもつ辺を含む DAG に対して、トポロジカルソートを用いる最短パスアルゴリズムを適用できるか答えよ。

8 グラフアルゴリズムの演習問題

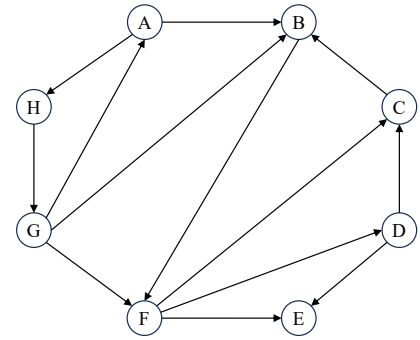
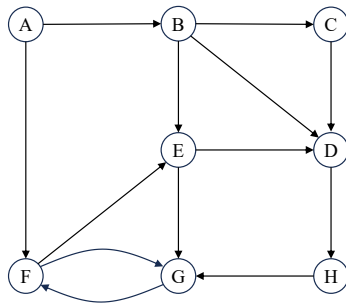
演習問題 8.1. 以下のグラフに対して深さ優先探索を実行せよ。ただし、未訪問の頂点から探索の始点を選ぶとき、および各頂点の隣接頂点を調べるときは、辞書順で最初の頂点を選ぶこと。各辺を、木辺または戻り辺に分類し、各頂点の pre と post の値を与えよ。



演習問題 8.2. 以下のグラフに対して深さ優先探索を実行せよ。ただし、未訪問の頂点から探索の始点を選ぶとき、および各頂点の隣接頂点を調べるときは、辞書順で最初の頂点を選ぶこと。各辺を、木辺・進み辺・戻り辺・渡り辺に分類し、各頂点の pre と post の値を与えよ。

演習問題 8.3. 有向グラフ $G = (V, E)$ の逆グラフ (reverse graph) $G^R = (V, E^R)$ とは、頂点集合が同じで辺が逆向きのグラフである。つまり、 $E^R = \{(v, u) \mid (u, v) \in E\}$ である。逆グラフの隣接リスト表現を線形時間で計算するアルゴリズムを与えよ。

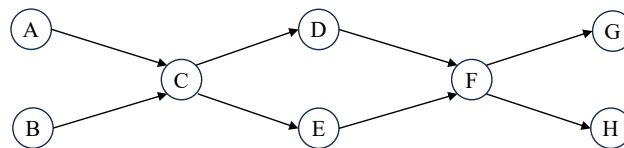
演習問題 8.4. 無向グラフの頂点 u の次数 (degree) $d(u)$ とは、 u に隣接する頂点の数であり、 u から出る



辺の数に等しい. 有向グラフでは, u に入る辺の数である**入次数** (*in-degree*) $d_{\text{in}}(u)$ と u から出ていく辺の数である**出次数** (*out-degree*) $d_{\text{out}}(u)$ を区別する.

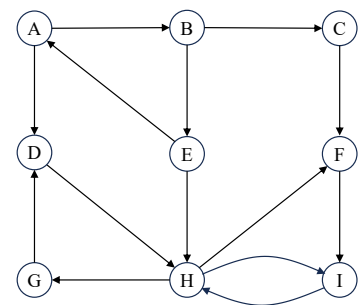
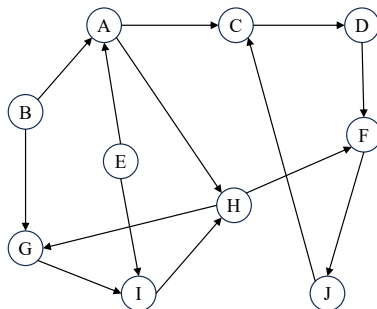
- (a) 無向グラフでは $\sum_{u \in V} d(u) = 2|E|$ であることを示せ.
- (b) (a) を利用して, 無向グラフでは次数が奇数である頂点数は偶数であることを示せ.
- (c) (b) と同様の主張が, 有向グラフにおける奇数入次数に関して成り立つか答えよ.

演習問題 8.5. 以下のグラフに対し, 深さ優先探索を利用してトポロジカルソートを見つけるアルゴリズムを実行せよ. ただし, 未訪問の頂点から探索の始点を選ぶとき, および各頂点の隣接頂点を調べるときは, 辞書順で最初の頂点を選ぶこと.



- (a) 各頂点の pre と post の値を与えよ.
- (b) グラフの始点と終点をすべて答えよ.
- (c) トポロジカルソートの結果を答えよ.
- (d) このグラフにはトポロジカルソートが何通りあるか答えよ.

演習問題 8.6. 以下の二つの有向グラフ G に対し, 強連結成分を見つけるアルゴリズムを実行せよ. G^R および G に対して深さ優先探索を実行するとき, 未訪問の頂点および隣接頂点は辞書順で最初のものを選ぶこと.



各グラフに対し以下を答えよ.

- (a) どの順番で強連結成分は見つかるか答えよ.
- (b) メタグラフの始点と終点に対応する強連結成分をすべて答えよ.
- (c) メタグラフを示せ.
- (d) このグラフを強連結にするには最低何本の辺を追加する必要があるか答えよ.

演習問題 8.7. 無向グラフに奇数の長さをもつ閉路が存在するかを線形時間で判定し、存在する場合にはそのような閉路を一つ出力するアルゴリズムを与えよ. (ヒント: まずはグラフが連結である場合を考えてみよう.)

演習問題 8.8. 無向グラフ G とその辺 e を入力とし、 G に e を含む閉路が存在するかどうかを判定する線形時間アルゴリズムを与えよ.

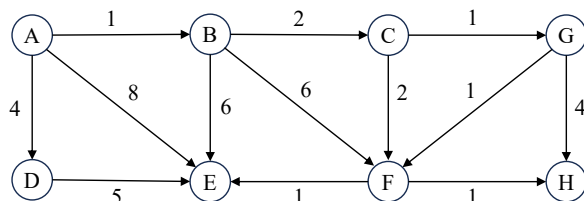
演習問題 8.9. 無向グラフの辺 (u, v) について、深さ優先探索の実行中に $\text{post}(u) < \text{post}(v)$ が成り立つとき、 v は深さ優先探索木において u の祖先である. これが正しいければ証明を、間違っていれば反例を与えよ.

演習問題 8.10. 有向グラフ $G = (V, E)$ を入力とし、すべての頂点 $v \in V$ へ到達可能な頂点 $s \in V$ が存在するかを判定する $O(|V| + |E|)$ 時間アルゴリズムを与えよ.

演習問題 8.11. DAG $G = (V, E)$ と 2 頂点 $s, t \in V$ を入力として受け取り、 s から t への異なる有向パスの数を出力する効率的なアルゴリズムを与えよ.

演習問題 8.12. DAG $G = (V, E)$ を入力として受け取り、 G の有向パスとして各頂点をちょうど一度だけ通るものが存在するかどうかを判定する線形計算量のアルゴリズムを与えよ.

演習問題 8.13. 以下のグラフに対し、頂点 A からダイクストラ法を実行したとする.



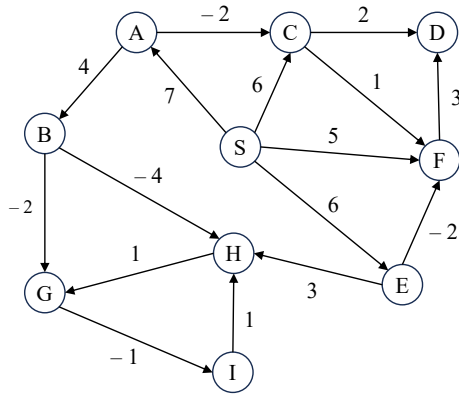
- (a) アルゴリズムの繰り返しにおいて A からの距離の表がどのように更新されていくか示せ.
- (b) 得られた最短パス木を示せ.

演習問題 8.14. 以下のグラフに対して、頂点 S を始点としてベルマン–フォード法を実行せよ. 各繰り返しでは、前回の dist の値だけを用いて同期的に更新するものとする. 各繰り返し後の dist の表と、得られた最短パス木を示せ.

演習問題 8.15. 無向グラフ $G = (V, E)$ に対し、 G を入力として受け取り、 G が長さ 4 の単純閉路を含むかどうかを判定するアルゴリズムを設計し、計算量を解析せよ. 計算量は $O(|V|^3)$ とすること. 入力グラフは隣接行列または隣接リスト表現のいずれかで与えられるものとする.

演習問題 8.16. 無向グラフ G が木であるとする. 二頂点 u, v を結ぶ単純パスを線形時間で出力せよ.

演習問題 8.17. 負閉路をもたない有向グラフとして、頂点 s から出る辺だけが負の重みをもち、その他の



辺はすべて正の重みをもつものを考える．このグラフに対してダイクストラ法を頂点 s から実行したとき、アルゴリズムは常に正しい最短距離を出力するか．自身の回答を証明せよ．

演習問題 8.18. 重み付き有向グラフ $G = (V, E)$ ，重み関数 $w : E \rightarrow \mathbb{R}$ ，および二頂点 $u, v \in V$ が与えられるとする． u から v への最短パスが存在し，その一つの辺数が高々 k であると仮定する． u から v への最短パスを $O(k|E|)$ 時間で見つけるアルゴリズムを与えよ．

演習問題 8.19. 正重みの有向グラフを入力とし，グラフの最短閉路の長さを出力するアルゴリズムを与えよ（閉路を含まない場合は，そうであることを出力すること）．アルゴリズムの計算量は $O(|V|^3)$ とすること．

演習問題 8.20. 正の重みをもつ無向グラフ $G = (V, E)$ ，重み関数 w ，および辺 $e \in E$ を入力とし， e を含む最短の単純閉路の長さを求めるアルゴリズムを与えよ．そのような閉路が存在しない場合には，その旨を出力すること．アルゴリズムの実行時間は $O(|V|^2)$ とすること．

クイズの答え

クイズ 1.1

(a) は隣接行列が適している．指定された二頂点の間に辺があるかを $O(1)$ 時間で判定できる．(b) は隣接リストが適している．疎グラフのすべての辺を $O(|V| + |E|)$ 時間で調べることができる．

クイズ 2.1

隣接リストで与えられる場合は

$$O(|V| + |E|)$$

である．隣接行列で与えられる場合は，各頂点について行列の一行を調べるため，

$$O(|V|^2)$$

である．

クイズ 2.2

$$\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$$

である．頂点 u の探索が終了する前に，その子孫 v の探索が開始し，終了するためである．

クイズ 3.1

戻り辺である．与えられた大小関係より， v は DFS 木において u の先祖である．したがって，辺 (u, v) は子孫から先祖へ向かう辺である．

クイズ 4.1

正しい． G における u から v へのパスのすべての辺を逆向きにすると， G^R における v から u へのパスになる．したがって，二頂点が互いに到達可能であるという関係は，すべての辺を逆向きにしても変わらない．

クイズ 4.2

そのような二頂点は存在しない．もし C から C' へのパスと， C' から C へのパスがともに存在すれば， C と C' の頂点は互いに到達可能になる．したがって， C と C' は同じ強連結成分でなければならない．

クイズ 5.1

高々 2 種類である．ある整数 d について，キュー内の頂点の距離は d または $d + 1$ である．距離 d の頂点を処理したときに新しく追加される頂点の距離が $d + 1$ だからである．

クイズ 5.2

正しい．幅優先探索では，始点からの距離が小さい頂点から順に処理される．したがって，各頂点を初めて発見したときに得られる距離は最短距離であり，幅優先探索木におけるパスも最短パスである．

クイズ 6.1

適用できる．ダイクストラ法が正しく動作するために必要なのは，すべての辺重みが非負であることである．したがって，重み 0 の辺が含まれていても問題ない．

クイズ 7.1

検出しない。始点 s から到達できない頂点の距離は ∞ のままであり、その負閉路上の辺を緩和しても値は更新されない。ベルマン–フォード法で検出できるのは、 s から到達可能な負閉路である。

クイズ 7.2

適用できる。DAG には閉路がないため、負閉路も存在しない。トポロジカルソートの順に頂点を処理すれば、負の重みをもつ辺があっても最短距離を正しく計算できる。